

Delhi Chief Minister Grievance & Complaint Management Dashboard

Combined Master Plan: Product Strategy + Technical Architecture + MVP Execution

Executive Summary

This document proposes a **production-minded MVP** for a Delhi Chief Minister grievance dashboard that does more than track complaints. It creates a **governance intelligence layer** for the CM's office: a unified system that ingests complaints from citizens, departments, social channels, and future government APIs; routes them intelligently; prevents false closures; exposes officer workload and bottlenecks; and gives the CM a mobile-first command view of live ground reality.

The core idea is simple:

Do not build a complaint form. Build an accountability engine.

The platform is designed for Delhi's scale, operational complexity, multilingual usage, and government deployment constraints. It uses the **MERN + Next.js** stack for compatibility with the challenge requirement, but the architecture is organized around government realities: partial API availability, manual fallback workflows, auditability, anti-corruption controls, citizen verification, and future extensibility for new departments, complaint types, and integration partners.

The strongest differentiators are:

1. **Field Visit Mode** — when the CM is physically in an area, the system shows nearby unresolved complaints and hotspots within a geofenced radius.
2. **Citizen Veto on Closure** — no complaint is truly closed until the citizen confirms resolution.
3. **Duplicate Complaint Clustering** — multiple complaints about the same location and issue are merged into a master ticket.
4. **Officer Accountability Graph** — every complaint is mapped to a responsible officer, timeline, evidence trail, and closure authority.
5. **Fallback-First Design** — if external department APIs are unavailable, the platform still functions via an internal employee dashboard.
6. **Auto-Segregation Engine** — complaint text, voice, image metadata, and location are used to auto-route the grievance.

This document is structured to help a team build the MVP quickly, present it clearly to judges, and expand it into a deployable production system if selected.

1. Problem Statement

Delhi's grievance ecosystem has a structural problem, not just a software problem.

The real pain points

- Citizens do not know where to file issues.
- Complaints are often delayed, reassigned, or lost between departments.
- Officers can falsely mark issues as resolved.
- CM and senior leadership lack real-time visibility into ground reality.
- Existing systems can be fragmented across departments and channels.
- Many workflows depend on manual coordination, which creates corruption risk and weak accountability.
- Department APIs may be incomplete, inconsistent, or unavailable.
- Most complaint portals are not designed for non-technical staff.
- Citizen experience is often too complex for high adoption.

What the CM actually needs

The CM does not primarily need another complaint inbox. The CM needs:

- a live operational map,
- a way to inspect local hotspots instantly,
- an accountability chain for each grievance,
- proof that resolution actually happened,
- a department-level performance view,
- and the ability to intervene directly in urgent cases.

2. Strategic Product Positioning

Incorrect framing

- Complaint registration portal
- Ticket tracking website
- Issue reporting app

Correct framing

- Governance command center
- Civic accountability platform
- Public grievance intelligence system
- CM field-monitoring dashboard

This distinction matters because judges and decision-makers are likely to reward a system that solves **administrative control, trust, and operational clarity**, not just ticket creation.

3. Product Principles

The solution should follow these principles:

3.1 Zero-friction citizen filing

Citizens should be able to report issues quickly using:

- WhatsApp bot
- mobile web portal
- helpline-assisted logging
- optional social media ingestion

3.2 Mobile-first CM experience

The CM dashboard must be highly usable on a phone, especially during site visits.

3.3 Fallback-first government operations

The system must not fail if department APIs are unavailable.

3.4 Anti-fraud by design

False closures, fake updates, and manipulation must be hard.

3.5 Auditability everywhere

Every action must be logged, attributable, and searchable.

3.6 Extensible architecture

New complaint types, new departments, new officers, and new integrations should be configurable rather than hardcoded.

4. User Roles

The MVP should support at least the following roles:

4.1 Citizen

- Submit complaint
- Attach photo/video/audio
- Track status
- Confirm or reject closure
- Receive notifications

4.2 CM / CM Office

- View live dashboard
- View nearby complaints during visits
- Prioritize critical cases
- Drill down by district, department, and severity
- Assign spot directives

4.3 Department Head / Nodal Officer

- View all complaints assigned to department
- Reassign cases
- Monitor officer workload
- Escalate unresolved tickets

4.4 Field Officer / Employee

- View assigned complaints
- Accept ownership
- Add updates and evidence
- Request escalation or reclassification

4.5 Super Admin / System Admin

- Manage departments, officers, categories
- Configure routing rules
- Manage SLA thresholds
- Review audit logs

5. Best-Fit Tech Stack

The challenge explicitly asks for **MERN / Next.js compatibility**, so the architecture is designed around that requirement.

5.1 Recommended stack overview

Layer	Recommended Technology	Why it fits
Frontend	Next.js + React	Fast UI, server-side rendering, mobile-friendly, government dashboard ready
UI System	Tailwind CSS + ShadCN UI	Clean, accessible, fast to build, professional admin interfaces
Backend	Node.js + Express	MERN-compatible, fast API development, event-driven workflows

Layer	Recommended Technology	Why it fits
Database	MongoDB	Flexible schema for changing complaint categories and metadata
Cache	Redis	Fast dashboard queries, session handling, and rate control
Job Queue	BullMQ	Background processing for notifications, clustering, AI tasks
File Storage	S3 / MinIO	Secure storage for images, videos, and documents
Auth	JWT + Refresh Tokens + OTP	Practical for citizen and staff login flows
Maps	Leaflet / Mapbox	Geospatial complaints, heatmaps, geofenced CM visit mode
Charts	Recharts / ECharts	Fast, readable governance dashboards
Search	Mongo Atlas Search / Elasticsearch optional	Fast lookup by complaint text, officer, ward, or keyword
Deployment	Docker	Portable, production-friendly packaging
CI/CD	GitHub Actions	Automated build/test/deploy pipeline
Observability	Prometheus + Grafana + Sentry	Health monitoring, logs, error tracking

6. Why This Stack Is the Best Fit

6.1 Why Next.js instead of plain React

Next.js gives:

- server-side rendering for fast dashboard load times,
- route-based organization,
- API routes if needed,
- better mobile performance,
- and easier deployment for a polished MVP.

This is especially helpful for CM and admin dashboards that must load quickly under operational pressure.

6.2 Why MongoDB instead of a strict relational-only model

Complaint records are naturally heterogeneous.

Example:

- a water complaint has pressure, contamination, pipe burst details,
- a road complaint has pothole depth, road condition, traffic impact,
- a law-and-order complaint has threat level and response urgency.

MongoDB handles evolving complaint schemas well, which is ideal for a government MVP where future complaint types may be added without major schema migrations.

6.3 Why Redis is necessary

Leadership dashboards often refresh the same metrics repeatedly:

- open complaints,
- district heatmaps,
- SLA breach counts,
- unresolved critical issues,
- officer workloads.

Those should not recompute from MongoDB on every refresh. Redis stores the computed result for a short time, making the dashboard fast and reliable.

6.4 Why BullMQ matters

Many government workflows are asynchronous:

- send SMS after complaint submission,
- recheck citizen confirmation after closure,
- generate nightly reports,
- classify complaint text,
- sync with external APIs.

BullMQ moves these out of the user request path so the system stays responsive.

6.5 Why S3 / MinIO matters

Field evidence must include photos and supporting files. These should live in object storage, not inside the database.

Benefits:

- better performance,
 - lower database bloat,
 - easy access control,
 - easier retention policies.
-

7. Core System Concept

The platform should be thought of as three connected engines:

7.1 Intake Engine

Receives complaints from citizens and channels.

7.2 Routing + Accountability Engine

Classifies, assigns, escalates, and tracks responsibility.

7.3 CM Intelligence Engine

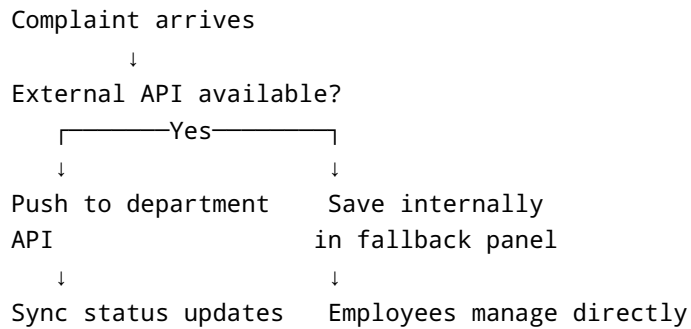
Shows live hotspot maps, officer load, department performance, and visit-mode insights.

8. End-to-End Workflow

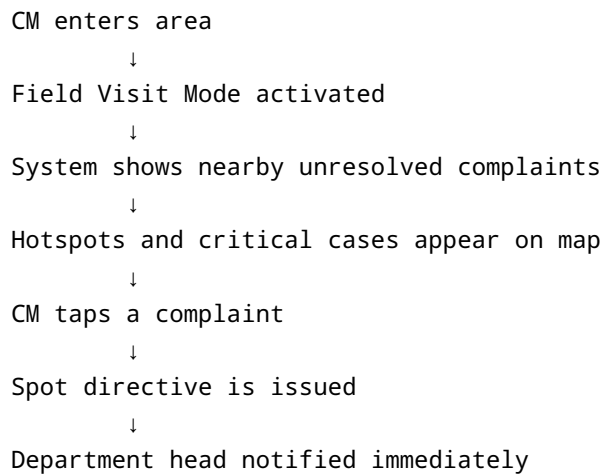
8.1 Citizen complaint flow

```
Citizen submits complaint
    ↓
Upload photo / voice / text / location
    ↓
System validates input
    ↓
Auto-segregation engine classifies issue
    ↓
Duplicate clustering checks for nearby open complaints
    ↓
Complaint is assigned to department / officer
    ↓
Citizen receives complaint ID and tracking link
    ↓
Officers work on ticket
    ↓
Evidence uploaded
    ↓
Citizen confirms or rejects closure
    ↓
Complaint closed only after verification
```

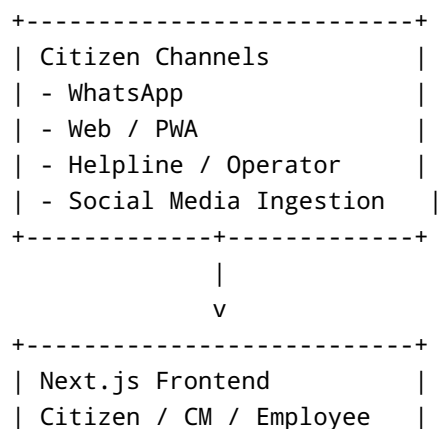
8.2 Department fallback flow

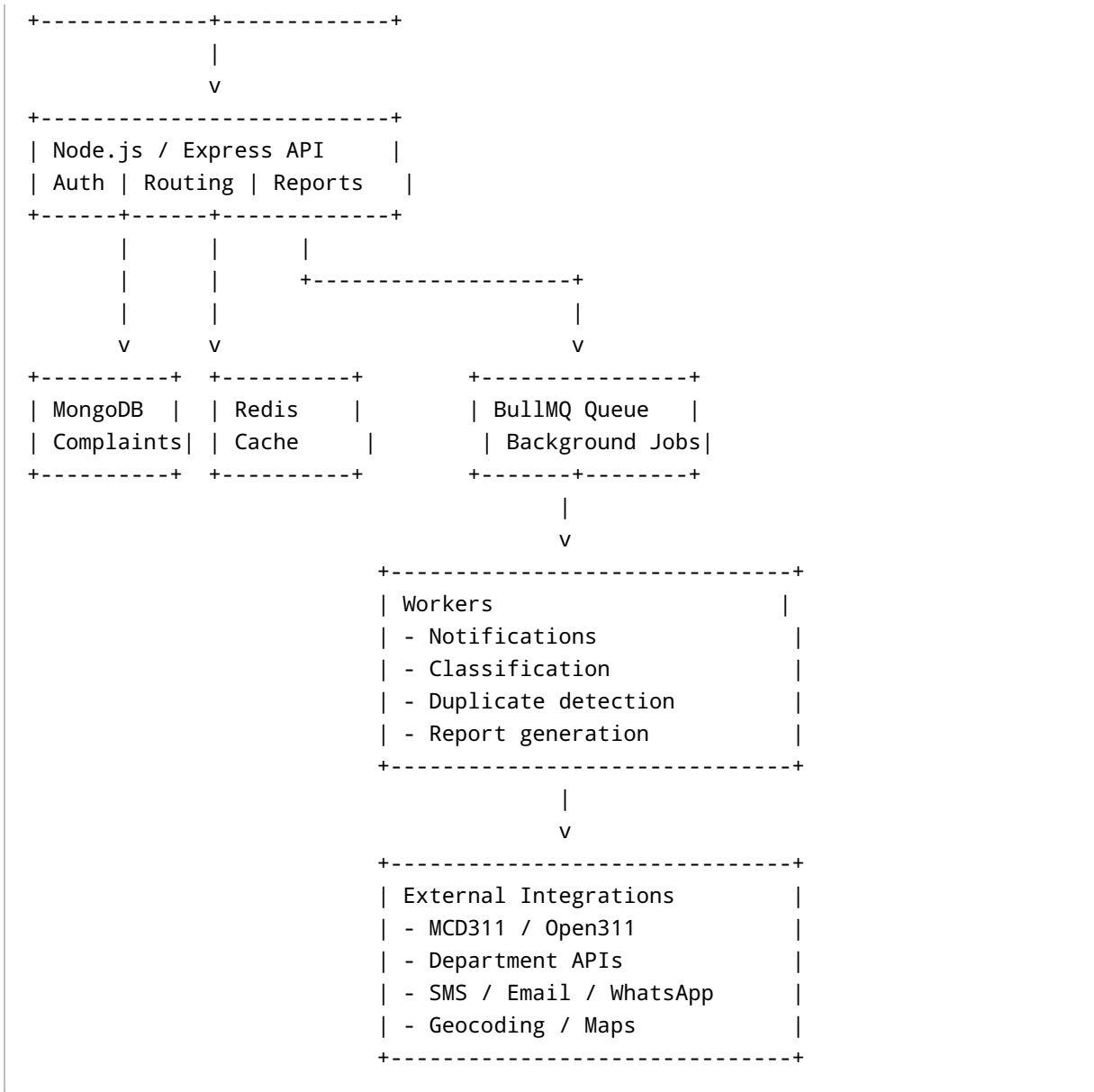


8.3 CM visit flow



9. Architecture Diagram





10. Data Model

Below is the minimum recommended database design.

10.1 Complaint collection

Field	Type	Purpose
<code>_id</code>	UUID/ string	Unique complaint ID

Field	Type	Purpose
referenceNumber	string	Citizen-facing tracking number
sourceChannel	enum	Web, WhatsApp, Helpline, Social, API
citizenId	string	Citizen identifier or phone hash
title	string	Short issue summary
description	text	Detailed issue report
category	string	High-level category
subCategory	string	More specific issue type
location	GeoJSON	Complaint coordinates
ward	string	Administrative ward
district	string	District name
priority	enum/int	Normal, High, Critical, Life-threatening
status	enum	Open, Assigned, In Progress, Provisionally Resolved, Closed, Reopened
departmentId	string	Assigned department
officerId	string	Assigned officer
clusterId	string	Master ticket ID if duplicated
attachments	array	Media URLs
timeline	array	Full action history
citizenVerification	object	Citizen approval/rejection status
createdAt	datetime	Timestamp
updatedAt	datetime	Timestamp

10.2 User collection

Field	Type	Purpose
_id	UUID/string	Unique user ID
name	string	Staff name
phone	string	Contact number
role	enum	Citizen, CM, Officer, DepartmentHead, Admin

Field	Type	Purpose
<code>departmentId</code>	string	User's department
<code>ward</code>	string	Operational jurisdiction
<code>designation</code>	string	Official designation
<code>isActive</code>	boolean	Access control
<code>lastLogin</code>	datetime	Audit record

10.3 Department collection

Field	Type	Purpose
<code>_id</code>	string	Department ID
<code>name</code>	string	Department name
<code>categoriesHandled</code>	array	Supported complaint types
<code>slaConfig</code>	object	SLA rules
<code>routingRules</code>	object	Assignment logic
<code>contactMatrix</code>	object	Nodal officer details
<code>apiConfig</code>	object	External API endpoint info

10.4 AuditLog collection

Field	Type	Purpose
<code>_id</code>	UUID/string	Log ID
<code>entityType</code>	string	Complaint/User/Department
<code>entityId</code>	string	Affected record
<code>action</code>	string	Created, Assigned, Updated, Closed, Reopened
<code>actorId</code>	string	Who performed action
<code>timestamp</code>	datetime	When it happened
<code>metadata</code>	object	Device, IP, reason, previous status

10.5 VisitLog collection

Field	Type	Purpose
<code>_id</code>	string	Visit log ID

Field	Type	Purpose
cmId	string	CM reference
location	GeoJSON	Visit location
startTime	datetime	Visit start
endTime	datetime	Visit end
notes	text	Observations
linkedComplaints	array	Nearby issues shown during visit

11. Complaint Routing Strategy

A good routing strategy should use a combination of rules and lightweight intelligence.

11.1 Rule-based routing

Examples:

- water shortage → Delhi Jal Board
- pothole → PWD / MCD depending on road type
- garbage overflow → MCD
- streetlight failure → relevant maintenance entity
- public safety critical issue → emergency escalations

11.2 AI-assisted routing

Use NLP to detect:

- language,
- intent,
- category,
- urgency,
- duplicate similarity.

This is not meant to be a heavy AI demo. It is meant to reduce manual sorting.

11.3 Geospatial routing

If a complaint has location coordinates, route based on:

- ward,
- district,
- department jurisdiction,

- proximity to known infrastructure assets if available.
-

12. Duplicate Complaint Clustering

This is a standout feature.

Problem

One broken water line can generate 50 to 200 complaints from the same street.

Solution

If the system sees a new complaint with:

- same category,
- same or nearby geolocation,
- similar text,
- and an open unresolved ticket already exists,

it should create a **master ticket** and attach the new complaint as a subscriber.

Benefits

- dashboard stays clean,
 - duplication does not distort analytics,
 - all citizens still receive updates,
 - officers work on one real issue instead of many copies.
-

13. False Closure Prevention System

This is one of the most important anti-corruption mechanisms.

13.1 Provisionally Resolved status

Officers should not immediately close a complaint. They should mark it as:

- **Provisionally Resolved**

13.2 Citizen verification step

Citizen receives:

- resolution photo,
- summary of action taken,

- buttons: Confirm / Reject.

13.3 Resolution logic

- If citizen confirms → complaint becomes Closed.
- If citizen rejects → complaint reopens and escalates.
- If citizen does not respond → supervisor review or auto-follow-up after a time threshold.

13.4 Evidence requirements

Closure should require:

- live field photo,
- geotag if available,
- timestamp,
- officer identity,
- reason for closure.

13.5 Why this matters

This makes false closures expensive and visible.

14. CM Field Visit Mode

This is one of the highest-value features for the CM view.

Goal

When the CM visits an area, the dashboard should behave like a live civic radar.

Functions

- show nearby unresolved issues within a geofenced radius,
- show complaint density around the CM's current location,
- highlight urgent or life-threatening issues,
- display nearby department load,
- allow immediate spot directives,
- store visit logs.

Why it stands out

This transforms the platform from a backend workflow tool into a real-time governance instrument.

15. Officer Resource Ledger

The dashboard should display a live matrix of officers and departmental bandwidth.

Minimum fields

- officer name,
- designation,
- department,
- assigned ward,
- open complaints count,
- overdue complaints,
- critical complaint count,
- workload percentage,
- status indicator,
- contact number.

Example view

Department	Officer	Ward	Open Tickets	Bandwidth	Status	Action
PWD	Executive Engineer	Karol Bagh	42	140%	Overloaded	Reassign
MCD	Assistant Commissioner	Rohini	12	45%	Available	Assign more

This helps the CM identify where the system is overloaded.

16. Critical Life-Threatening Complaint Alerts

Some complaints should bypass normal queues.

Examples

- open manhole on a main road,
- exposed live wire,
- severe waterlogging under a bridge,
- gas leak,
- fire-related public hazard,
- collapsed structure risk.

Behavior

- instant red alert on CM and officer dashboards,

- SMS/WhatsApp escalation,
 - mandatory acknowledgement,
 - aggressive SLA timer,
 - automatic leadership notification if not acknowledged.
-

17. Citizen Channels

17.1 WhatsApp bot

Best for adoption and convenience.

Flow:

- citizen says hello,
- bot asks for photo, location, and short description,
- system creates ticket,
- citizen gets ticket ID.

17.2 Mobile-first web portal

- zero-login or OTP login,
- simple complaint form,
- upload media,
- track status.

17.3 Social media ingestion

Optional but valuable.

- citizen tags official handle,
- system drafts complaint,
- admin verifies and converts it into a ticket.

17.4 Helpline/operator entry

For citizens without digital comfort.

- operator records complaint into the same system.
-

18. Admin and Employee Dashboard Design

The admin side must be simple.

Required usability principles

- big buttons,
- clear labels,
- minimal technical jargon,
- color-coded urgency,
- fast search,
- no clutter,
- Kanban-like workflow visibility.

Key views

- New complaints
- Assigned complaints
- In progress
- Awaiting citizen verification
- Escalated
- Closed

Why this matters

Non-technical government staff need a system they can learn quickly. A dashboard with too much complexity will fail in real deployment.

19. Analytics and CM Intelligence Layer

The CM dashboard should answer questions like:

- Where are complaints concentrated?
- Which department is slow?
- Which officer has too many unresolved cases?
- Which ward has recurring issues?
- Which complaints are emerging as hotspots?
- Where should the CM intervene personally?

Core charts

- total complaints by district,
- complaints by category,
- SLA compliance rate,
- open vs closed trend,
- average resolution time,
- repeated complaints by location,
- critical alerts timeline,
- officer workload chart.

Geospatial analytics

- heatmap by ward,
 - cluster view,
 - nearest unresolved complaints,
 - recurring local issues.
-

20. Security Architecture

Government systems require a higher standard of security and accountability.

20.1 Authentication

- OTP-based login for citizens,
- secure login for officers and admin users,
- role-based access control,
- session expiration,
- refresh token rotation.

20.2 Authorization

Each user should only see the data they are allowed to see.

20.3 Data protection

- encrypt sensitive data at rest,
- use HTTPS everywhere,
- avoid unnecessary personal data collection,
- store only what is needed,
- redact sensitive fields in logs.

20.4 Audit logging

Every major action should be tracked:

- complaint creation,
- reassignment,
- closure,
- reopening,
- verification,
- edit attempts,
- login events.

20.5 Anti-manipulation logic

The system should detect suspicious patterns such as:

- repeated same-day closures by the same officer,
 - frequent reopenings from the same area,
 - missing evidence uploads,
 - abnormal resolution times,
 - high rejection rate of citizen confirmations.
-

21. Fallback-First Integration Strategy

This is essential for government deployment.

Rule

The platform must continue to work even if external department APIs are missing or unstable.

Integration behavior

- attempt API sync where available,
- if API succeeds, keep two-way sync,
- if API fails, store complaint internally,
- provide a native employee dashboard for manual handling,
- allow later reconciliation with department systems.

Why this is strong

Many systems fail because they assume full API maturity. This design does not.

22. API Design

22.1 Core endpoints

- `POST /api/complaints`
- `GET /api/complaints/:id`
- `PATCH /api/complaints/:id/status`
- `PATCH /api/complaints/:id/assign`
- `POST /api/complaints/:id/verify`
- `GET /api/dashboard/cm-summary`
- `GET /api/dashboard/heatmap`
- `GET /api/dashboard/officer-load`
- `POST /api/integrations/sync`

22.2 Design principles

- simple REST structure,
 - validation on all inputs,
 - versioned routes,
 - clear error codes,
 - idempotency where needed.
-

23. MVP Feature Priority

The MVP should focus on high-impact features only.

Must-have

- citizen complaint intake,
- WhatsApp or web intake,
- auto-routing,
- officer dashboard,
- CM dashboard,
- duplicate clustering,
- citizen verification,
- audit logs,
- fallback employee panel,
- map + hotspot analytics.

Good-to-have

- social media ingestion,
- voice input,
- multilingual NLP classification,
- visit mode,
- officer resource ledger,
- live notifications.

Not required for MVP

- overly complex ML models,
 - full mobile apps for every user role,
 - deep integration with every department from day one,
 - expensive cloud vendor lock-in.
-

24. Proposed MVP Sprint Plan

Sprint 1: Foundation

- repository setup,
- Next.js frontend scaffold,
- Node/Express API scaffold,
- MongoDB schema design,
- basic auth.

Sprint 2: Citizen intake

- complaint submission form,
- media uploads,
- OTP verification,
- complaint ID generation,
- notification after submission.

Sprint 3: Routing + employee fallback

- department mapping logic,
- officer assignment,
- fallback dashboard,
- status update flow.

Sprint 4: CM dashboard

- summary cards,
- map view,
- filters,
- charts,
- critical alerts.

Sprint 5: Anti-fraud + analytics

- citizen verification on closure,
- duplicate clustering,
- audit logs,
- officer performance metrics.

Sprint 6: Polish + demo readiness

- mobile optimization,
- multilingual support,
- seeded demo data,
- test scenarios,
- presentation deck.

25. Team Structure for Maximum Output

For a team of five:

Member	Focus
1	Next.js UI + dashboards
2	Node.js APIs + auth
3	MongoDB + data model + audit logs
4	Maps, analytics, charts, duplicate clustering
5	Notifications, integrations, WhatsApp/social/voice flows

26. Demo Strategy

The demo should be built around a story, not a feature dump.

Demo narrative

1. Citizen reports an issue.
2. System auto-routes it.
3. Officer receives assignment.
4. CM sees it on the dashboard.
5. CM visits the area and views nearby grievances.
6. Officer uploads evidence.
7. Citizen confirms or rejects closure.
8. Dashboard updates accountability metrics.

Why this works

It proves the system is not just a form. It proves the system is a governance loop.

27. Judging Strategy

The system should clearly answer the likely judging criteria:

27.1 Idea quality

We are solving the CM's actual governance pain point: truth, control, and accountability.

27.2 Ease of use

Citizens get a simple intake path; staff get a minimal admin panel.

27.3 Innovation

Field Visit Mode, citizen veto, duplicate clustering, and accountability graph stand out.

27.4 Technical completeness

Fallback mode ensures the system is practical even without external APIs.

27.5 Scalability and future readiness

The schema, routing, and role model are built for expansion.

28. Risks and Mitigation

Risk	Impact	Mitigation
Government APIs unavailable	Integration delays	Use fallback employee dashboard
Too many duplicate complaints	Dashboard clutter	Master ticket clustering
False closures	Corruption risk	Citizen veto + proof upload
Poor citizen adoption	Low usage	WhatsApp + PWA + helpline
Non-technical staff struggle	Operational failure	Simplified UI + Kanban workflow
Data growth	Slower dashboards	Redis cache + indexing
Security concerns	Trust failure	RBAC, logging, encryption

29. Recommended Presentation Narrative

A strong presentation should say:

Delhi does not need another complaint form. It needs a live accountability system.

Our dashboard unifies citizen complaints, department workflows, CM field visibility, and anti-fraud resolution controls into one platform.

It works even when APIs are missing, it scales with Delhi's size, and it gives the CM a way to see what is actually happening on the ground.

That framing is more compelling than “we built a MERN app.”

30. The Best Differentiators to Emphasize

If the team wants to stand out, emphasize these five:

1. **Field Visit Mode** — geo-aware complaint visibility for CM site visits.
2. **Citizen Veto** — closure only after citizen confirmation.
3. **Duplicate Master Tickets** — removes complaint spam and improves analytics.
4. **Fallback Employee Dashboard** — works even without APIs.
5. **Accountability Graph** — complaint-to-officer traceability.

These features combine political usefulness, operational realism, and technical credibility.

31. Suggested Final Product Name Ideas

- **Delhi Grievance Command Center**
- **CM Accountability Dashboard**
- **Delhi Civic Intelligence System**
- **Grievance OS for Delhi**
- **Delhi 360 Governance Dashboard**

The best name is one that sounds serious, government-ready, and operational.

32. Final Recommendation

Build the MVP as a **government-grade grievance operating system** with:

- **Next.js + React** for the frontend,
- **Node.js + Express** for backend APIs,
- **MongoDB** for flexible complaint records,
- **Redis + BullMQ** for speed and background workflows,
- **S3 / MinIO** for evidence storage,
- **role-based dashboards** for CM, staff, and citizens,
- **citizen confirmation before final closure**,
- **geospatial intelligence** for map-based decision-making,
- **fallback workflows** for API failures,
- and **audit logs** for every meaningful action.

If the team builds this well, it will not look like a student project. It will look like something a government office could realistically pilot.

Appendix A: One-Line Technical Pitch

A mobile-first, audit-heavy, fallback-first grievance intelligence platform for Delhi that unifies citizens, officers, and the CM into a single accountability loop.

Appendix B: One-Line Founder Pitch

We built the system that lets the Chief Minister see complaints, officers, and ground truth in one place — and prevents fake closures from disappearing into the bureaucracy.